

CineMax

Manuale Tecnico

Laboratorio Interdisciplinare A – A.A. 2025/2026

Alessandro Panarotto – matricola 757930 – sede VA

Federico Trentini – matricola 760478 – sede VA

Mohan Thomas Paolo – matricola 761573 – sede VA

Davide Paolo Calabrese – matricola 763012 – sede VA

1. Introduzione

Questo documento descrive le scelte architetturali, algoritmiche e le strutture dati adottate nello sviluppo di CineMax, l'applicazione a riga di comando per la gestione di un cinema monosala (proiezioni, prenotazioni e utenti con tre ruoli: cliente, proiezionista, bigliettaio). Per le istruzioni di installazione, compilazione ed esecuzione si rimanda al file README.txt incluso nella consegna; per la descrizione delle funzionalita' dal punto di vista dell'utente finale si rimanda al Manuale Utente.

Il progetto e' scritto in Java standard, senza librerie esterne (la cartella lib/ e' vuota): tutta la logica e' implementata con le sole API del JDK (java.io, java.time, java.security, java.util). I dati sono persistiti su file di testo, non tramite serializzazione di oggetti, per renderli leggibili e ispezionabili anche manualmente.

2. Architettura generale

Il codice e' diviso in package, e ogni package si occupa di una cosa precisa: ognuno parla solo con quello "sopra" o "sotto" di lui, senza saltare livelli:

```
cinemax (classe main, punto di ingresso)
cinemax.ui (menu testuali: input/output)
cinemax.service (logica applicativa e regole di business)
cinemax.data (persistenza: lettura/scrittura dei file di testo)
cinemax.model (le entita' del dominio: Utente, Proiezione, ...)
cinemax.eccezioni (eccezioni custom, usate in tutti i livelli)
cinemax.util (classi di utilita' trasversali)
```

Il flusso tipico di una richiesta e' sempre lo stesso, indipendentemente dal ruolo o dalla funzionalita':

```
Menu (input/output)
-> Service (applica le regole, lancia eccezioni se violate)
-> Gestore/Data (legge/scrive il file di testo)
-> Model (rappresenta il dato in memoria)
```

Perche' abbiamo diviso il codice cosi'. Se cambia il formato di un file, si tocca solo il Gestore corrispondente. Se cambia una regola (es. quanti biglietti si possono prenotare), si tocca solo il Servizio. Se cambia come viene mostrato un menu, si tocca solo la classe Menu. I Menu non contengono regole, e i Servizi non leggono o scrivono mai direttamente i file: in questo modo, quando uno di noi modificava una parte del programma, gli altri non dovevano toccare le proprie classi.

Lo stesso schema a livelli si ripete identico per le tre entita' principali del sistema (proiezioni, prenotazioni, utenti), ciascuna con il proprio Gestore, il proprio Servizio e i propri Menu dedicati ai ruoli che la utilizzano.

3. Modello dei dati (cinemax.model)

3.1 Gerarchia degli utenti

Utente e' una classe abstract che raccoglie i dati comuni a tutti gli utenti registrati (nome, cognome, username, password cifrata, data di nascita facoltativa, domicilio facoltativo) e dichiara il metodo abstract **getRuolo()**. Le tre sottoclassi **Cliente**, **Proiezionista** e **Bigliettaio** estendono Utente senza aggiungere alcun campo proprio: si limitano a fare l'override di **getRuolo()**, restituendo il valore corrispondente dell'enum Ruolo.

Perche' una classe abstract. Così non dobbiamo riscrivere gli stessi campi (nome, cognome, username...) in tre classi diverse, e il **GestoreUtenti** può gestire una semplice **List<Utente>** senza doversi preoccupare di quale ruolo abbia ogni utente. Il ruolo specifico serve solo in pochi punti, ad esempio nella classe **main**, dove un controllo **instanceof** decide quale Menu aprire dopo il login.

3.2 Le altre entita'

- **Film** – titolo, genere, regista, anno, durata in minuti, eta' minima. E' un dato indipendente, referenziato da una o piu' **Proiezione**.
- **Proiezione** – id univoco (final), riferimento a un oggetto **Film**, data/ora, costo del biglietto. Contiene il metodo **siSovrapponeCon()**, descritto nella sezione 5, che e' l'algoritmo piu' importante del package **model**.
- **Prenotazione** – codice univoco (final), *idProiezione* e *usernameCliente* (non riferimenti diretti agli oggetti **Proiezione/Cliente**, ma alle loro chiavi), numero di biglietti.
- **Ruolo** – enum con i tre valori **CLIENTE**, **PROIEZIONISTA**, **BIGLIETTAIO**.

Perche' chiavi e non riferimenti ad oggetti. **Prenotazione** non mantiene un riferimento Java diretto alla **Proiezione** o al **Cliente** a cui si riferisce, ma solo il loro identificativo testuale (id numerico, username). Questa scelta e' diretta conseguenza del fatto che i dati vengono salvati come testo piatto: un riferimento ad oggetto non avrebbe senso su file, mentre un id si scrive e si rilegge in una singola riga. Il costo del biglietto, inoltre, non viene duplicato nella **Prenotazione**: viene sempre recuperato al momento della visualizzazione interrogando la **Proiezione** collegata, così se il prezzo cambia non si creano incongruenze tra i dati.

4. Persistenza dei dati (cinemax.data)

Le classi **GestoreUtenti**, **GestoreProiezioni** e **GestorePrenotazioni** si occupano esclusivamente di leggere/scrivere, rispettivamente, i file **data/utenti.txt**, **data/proiezioni.txt** e **data/prenotazioni.txt**. Nessun'altra classe del progetto accede direttamente a questi file.

4.1 Formato dei file

Ogni file e' un testo semplice con una entita' per riga, campi separati da punto e virgola (;); le righe vuote e quelle che iniziano con '#' (usate come intestazione descrittiva) vengono ignorate in lettura.

File	Formato riga
utenti.txt	ruolo;nome;cognome;username;passwordCifrata;dataNascita;domicilio
proiezioni.txt	id;titolo;genere;regista;anno;durataMinuti;etaMinima;dataOra;costoBiglietto
prenotazioni.txt	codice;usernameCliente;idProiezione;numeroBiglietti

4.2 Strategia di lettura/scrittura

- **Caricamento unico all'avvio:** ogni Servizio (vedi sezione 5) chiama il proprio Gestore una sola volta, nel costruttore, e mantiene la lista risultante in memoria (`List<T>`, `ArrayList`) per tutta la durata dell'esecuzione.
- **Riscrittura completa ad ogni modifica:** i metodi `salvaXxx()` non fanno `append`, ma sovrascrivono interamente il file con il contenuto aggiornato della lista in memoria. E' una scelta semplice e robusta per file di queste dimensioni (centinaia/migliaia di righe), che evita incongruenze tra cio' che e' in memoria e cio' che e' su disco.
- **Parsing difensivo:** ogni Gestore prova a interpretare la riga in un blocco `try/catch`; se una riga e' malformata (campi mancanti o non convertibili) viene scartata con un messaggio di avviso in console, invece di interrompere il caricamento di tutto il file. Questo rende il programma tollerante a errori di formattazione isolati nei file dati.

Perche' file di testo e non serializzazione. Si e' scelto deliberatamente di non usare `ObjectOutputStream/ObjectInputStream` (serializzazione binaria degli oggetti Java). I file di testo restano leggibili e modificabili anche con un semplice editor, sono indipendenti dalla versione delle classi Java (un campo serializzato legato a una versione precisa della classe puo' rompersi cambiando il codice), e corrispondono al formato richiesto dalle specifiche di progetto (il file `proiezioni.csv` fornito dal docente come dato di partenza).

5. Logica applicativa (cinemax.service)

Le tre classi `ServizioProiezioni`, `ServizioPrenotazioni` e `ServizioUtenti` contengono tutte le regole di business del progetto. Sono le uniche classi, oltre ai Gestore, a manipolare le liste in memoria, e sono le uniche a lanciare le eccezioni custom descritte nella sezione 6.

5.1 Controllo delle sovrapposizioni tra proiezioni

Il cinema gestito da CineMax e' monosala: non puo' esserci piu' di un film in programmazione nello stesso momento. Il controllo e' implementato nel metodo `Proiezione.siSovrapponeCon(Proiezione altra)`, con la classica condizione di sovrapposizione tra due intervalli temporali:

```
inizio1 < fine2 AND inizio2 < fine1
```

dove `fine1/fine2` sono calcolati come `dataOra + durata del film (getDataOraFine())`. `ServizioProiezioni.controllaSovrapposizione()` scorre linearmente tutte le proiezioni esistenti (escludendo, in caso di modifica, la proiezione stessa) e lancia `ProiezioneSovrappostaException` se trova un conflitto. Questo controllo viene eseguito sia in `aggiungiProiezione()` che in `modificaProiezione()`.

5.2 Calcolo dei posti liberi

`ServizioProiezioni.postiLiberi(proiezione, tuttePrenotazioni)` calcola `Costanti.CAPIENZA_SALA` meno la somma dei `numeroBiglietti` di tutte le prenotazioni collegate a quella proiezione (confronto sull'`idProiezione`). Il risultato viene usato sia per mostrare la disponibilita' (`visualizzaProiezione`) sia, in `ServizioPrenotazioni`, per validare una nuova prenotazione o la modifica di una esistente prima di accettarla.

5.3 Cosa succede se una modifica crea un conflitto

Quando si chiama `modificaProiezione()`, il metodo applica subito i nuovi valori (data/ora e/o costo) e poi controlla se la proiezione si sovrappone a qualche altra. Se c'è un conflitto, **i valori vecchi vengono rimessi com'erano** prima di lanciare `ProiezioneSovrappostaException`, così la proiezione resta com'era prima del tentativo di modifica fallito, invece di restare a metà modificata.

5.4 Ricerca con criteri opzionali

`cercaProiezione()` e `cercaPrenotazione()` accettano diversi parametri di filtro, tutti facoltativi (possono essere null o stringa vuota). Ogni criterio non nullo viene applicato in AND logico con gli altri; un criterio nullo viene semplicemente saltato. Questo permette di implementare con un solo metodo sia la ricerca senza filtri (tutte le proiezioni) sia ricerche molto specifiche (es. titolo parziale + intervallo di date + intervallo di prezzo), senza dover scrivere metodi diversi per ogni combinazione.

5.5 Vincoli temporali su modifica ed eliminazione delle prenotazioni

Le specifiche di progetto richiedono due vincoli temporali opposti, implementati letteralmente in `ServizioPrenotazioni`:

- **`modificaPrenotazione()`** è permessa solo se sia la proiezione originale sia la nuova proiezione scelta hanno una data/ora *futura* rispetto al momento dell'operazione (altrimenti `DataNonValidaException`).
- **`eliminaPrenotazione()`** è permessa solo se la proiezione collegata ha una data/ora *passata* rispetto al momento dell'operazione.

Analogamente, `ServizioProiezioni` vieta la modifica o l'eliminazione di una proiezione se esistono prenotazioni collegate (`PrenotazioneEsistenteException`), per non invalidare biglietti già venduti a un cliente.

5.6 Generazione di identificativi univoci

Gli id delle proiezioni sono numeri interi progressivi: il prossimo id è calcolato come massimo tra gli id esistenti più uno. I codici delle prenotazioni, invece, sono alfanumerici e vengono generati da `CodiceUtil.generaCodicePrenotazione()` combinando il prefisso "PRN", il timestamp corrente in millisecondi e un contatore `AtomicInteger` incrementato ad ogni chiamata: questo garantisce un codice diverso anche se due prenotazioni vengono create nello stesso millisecondo.

6. Gestione delle eccezioni (cinemax.eccezioni)

Il progetto definisce sette eccezioni personalizzate, tutte **checked** (estendono `Exception` e non `RuntimeException`): il compilatore obbliga quindi ogni chiamante a gestirle esplicitamente con `try/catch` o a propagarle con `throws`. La scelta di renderle checked è deliberata: si tratta di violazioni di regole di business previste dalle specifiche (non di errori di programmazione), quindi è corretto che il codice sia obbligato a prevederle e gestirle in modo esplicito.

Eccezione	Lanciata quando...
<code>CredenzialiErrateException</code>	username o password non corrispondono durante il login
<code>UsernameEsistenteException</code>	in fase di registrazione l'username scelto è già in uso

ProiezioneSovrappostaException	una proiezione nuova/modificata si sovrappone in orario a una esistente
PrenotazioneEsistenteException	si tenta di modificare/eliminare una proiezione con prenotazioni collegate
PostiNonDisponibiliException	si richiedono piu' biglietti di quanti posti siano liberi
DataNonValidaException	una data non rispetta i vincoli richiesti (modifica/eliminazione prenotazione)
ElementoNonTrovatoException	si cerca un elemento (proiezione, prenotazione) tramite un id/codice inesistente

Come vengono gestite. L'eccezione viene lanciata dal Servizio, dove si trova la regola che e' stata violata, e viene catturata dal Menu, che la trasforma in un messaggio leggibile con `e.getMessage()` invece di far vedere uno stack trace o bloccare il programma. Anche gli errori di lettura/scrittura dei file (`IOException`) vengono catturati nello stesso modo, sempre nel Menu, e mostrati come un messaggio semplice.

7. Interfaccia utente testuale (cinemax.ui)

Ogni ruolo ha una propria classe Menu (`MenuGuest`, `MenuCliente`, `MenuProiezionista`, `MenuBigliettaio`), avviata dalla classe main dopo il login (o subito, per il guest). Tutti seguono lo stesso schema: un ciclo while che mostra le opzioni numerate e legge la scelta con `InputUtil`, e uno switch che invoca il metodo privato corrispondente.

La classe `MenuUtil` raccoglie le funzioni di stampa/selezione condivise da piu' menu (`stampaElencoProiezioni`, `stampaDettagliProiezione`, `chiediCriteriERicerca`, `selezionaProiezione`), evitando di duplicare lo stesso codice in `MenuGuest`, `MenuCliente` e `MenuProiezionista`.

Nei Menu non ci sono regole. Ogni metodo dei Menu fa solo tre cose: (1) chiede i dati all'utente, (2) chiama il metodo del Servizio dentro un blocco try/catch, (3) stampa il risultato o il messaggio dell'eccezione. Se in futuro cambiasse una regola (es. il vincolo sulle date per eliminare una prenotazione), basterebbe modificare `ServizioPrenotazioni`, senza toccare i Menu.

8. Classi di utilita' (cinemax.util)

Classe	Responsabilita'
Costanti	valori condivisi: capienza sala, percorsi dei tre file dati, formati di stampa delle date
InputUtil	lettura validata da tastiera (stringhe, interi, decimali, date, si/no): ripete la domanda finche' il valore non e' valido
PasswordUtil	hashing SHA-256 one-way delle password prima del salvataggio e verifica al login
CodiceUtil	generazione di id progressivi (proiezioni) e codici univoci (prenotazioni)

Tutte queste classi sono *final* con costruttore privato e contengono solo membri *static*: non rappresentano un oggetto del dominio, ma una raccolta di funzioni di supporto, quindi non ha

senso poterle istanziare.

Sicurezza delle password. PasswordUtil non salva mai la password in chiaro: calcola l'hash SHA-256 (funzione one-way, non invertibile) e salva solo l'hash esadecimale. Al login si ricalcola l'hash della password digitata e si confrontano le due stringhe, senza mai decifrare nulla.

9. Strutture dati utilizzate

Struttura	Dove e perche'
List (ArrayList)	lista in memoria di Utente/Proiezione/Prenotazione in ciascun Servizio; rispecchia il contenuto del file di testo corrispondente e viene riscritta per intero ad ogni modifica
AtomicInteger	contatore usato da CodiceUtil per generare codici di prenotazione sempre diversi, anche in caso di richieste molto vicine nel tempo
LocalDate / LocalDateTime	(java.time) per le date di nascita, le date/ora delle proiezioni e tutti i confronti temporali (sovrapposizioni, vincoli su modifica/eliminazione)
Riferimenti per chiave (id/username/codice)	tra Prenotazione e Proiezione/Cliente, invece di riferimenti diretti ad oggetti Java: necessario per la persistenza su file di testo piatto
enum Ruolo	i tre ruoli del sistema, usati sia nel model (Utente.getRuolo()) sia nella persistenza (primo campo di ogni riga di utenti.txt)

Una nota. Le ricerche (controllo sovrapposizione, calcolo posti liberi, ricerca per id/username/codice) scorrono semplicemente tutta la lista elemento per elemento, senza usare strutture piu' complesse come mappe o alberi. Per le quantita' di dati previste dal progetto va benissimo, ed e' molto piu' semplice da scrivere e da capire.

10. Classe main (cinemax.CineMax)

CineMax.main() e' l'unico punto di ingresso del programma. Crea i tre Servizi (ServizioUtenti, ServizioProiezioni, ServizioPrenotazioni), terminando con un messaggio d'errore se il caricamento iniziale dei file dati fallisce, poi mostra il menu principale: login, registrazione di un nuovo cliente, oppure accesso come visitatore (guest) per cercare un film senza autenticarsi. Dopo un login riuscito, un controllo instanceof sull'oggetto Utente restituito da ServizioUtenti.login() determina quale Menu specifico avviare (Cliente, Proiezionista o Bigliettaio).

11. Considerazioni finali

11.1 Estendibilita'

Grazie alla separazione in livelli, alcune evoluzioni future del progetto richiederebbero modifiche localizzate a un solo package:

- sostituire la persistenza su file con un database relazionale: basterebbe riscrivere le classi del package cinemax.data, lasciando invariati model, service e ui;

- aggiungere un nuovo ruolo utente: basterebbe una nuova sottoclasse di Utente, un nuovo valore nell'enum Ruolo e un nuovo Menu dedicato, senza toccare le classi Servizio esistenti;
- aggiungere un nuovo criterio di ricerca: basterebbe aggiungere un parametro opzionale al metodo cercaProiezione()/cercaPrenotazione() del Servizio interessato.

11.2 Limiti noti

- il cinema gestito e' monosala: il controllo di sovrapposizione assume un'unica sala di proiezione, come esplicitamente previsto dalle specifiche;
- la riscrittura completa del file ad ogni modifica non e' transazionale: in caso di interruzione del programma durante la scrittura, il file potrebbe risultare incompleto. Per il contesto del progetto (uso singolo utente da riga di comando) questo rischio e' stato considerato accettabile;
- le operazioni di ricerca sono $O(n)$: adeguate al volume di dati del progetto, ma non pensate per dataset di dimensioni molto maggiori.